CHAPTER 9

SAMPLE CODE

This chapter presents the main source files of the Forensic Face Generation and Matching System in reading order, starting with the configuration layer and progressing through the pipeline modules (speech-to-text, attribute parsing, face generation, mark rendering, ONNX face processing, and two-layer matching) to the Flask application, extensions and database schema. Every block below is taken directly from the working code base.

9.1 Configuration

File: `config.py`

```
"""
Forensic Combined System — Configuration
Merges System A (AI pipeline) + System B (Flask/DB) settings
"""
import os
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

# ── Paths ──────────────────────────────────────────────────────
BASE_DIR        = Path(__file__).parent
DATA_DIR        = BASE_DIR / "data"
CRIMINAL_DB_DIR = DATA_DIR / "criminal_db"
GENERATED_DIR   = DATA_DIR / "generated_faces"
UPLOADS_DIR     = DATA_DIR / "uploads"
MODELS_DIR      = BASE_DIR / "models"
DATABASE_PATH   = DATA_DIR / "forensic.db"

for d in [CRIMINAL_DB_DIR, GENERATED_DIR, UPLOADS_DIR, MODELS_DIR]:
    d.mkdir(parents=True, exist_ok=True)

# ── Flask ──────────────────────────────────────────────────────
SECRET_KEY              = os.getenv("SECRET_KEY", "change-this-in-production-please")
SESSION_LIFETIME_HOURS  = 8
MAX_UPLOAD_MB           = 16

# ── AI APIs ────────────────────────────────────────────────────
DEEPGRAM_API_KEY  = os.getenv("DEEPGRAM_API_KEY", "")
DEEPGRAM_MODEL    = "nova-2"
DEEPGRAM_LANGUAGE = "en"

# Cloudflare Workers AI — free tier (10,000 neurons/day) hosts
# FLUX.1-schnell for forensic face generation.
CLOUDFLARE_ACCOUNT_ID = os.getenv("CLOUDFLARE_ACCOUNT_ID", "")
CLOUDFLARE_API_TOKEN  = os.getenv("CLOUDFLARE_API_TOKEN",  "")
```

```
# ── Face Generation ──────────────────────────────────────────
# GENERATION_MODE supports exactly two values:
#   "local" → SD 1.5 Realistic Vision + LCM-LoRA on this machine.
#             ControlNet sketch-to-photo included. Required for sketch uploads.
#   "api"   → FLUX.1-schnell via Cloudflare Workers AI or Pollinations.
GENERATION_MODE = os.getenv("GENERATION_MODE", "local")

USE_CLOUDFLARE = os.getenv("USE_CLOUDFLARE", "true").strip().lower() in (
    "1", "true", "yes", "on",
)

CLOUDFLARE_IMAGE_MODEL = "@cf/black-forest-labs/flux-1-schnell"
CLOUDFLARE_API_BASE    = "https://api.cloudflare.com/client/v4/accounts"
CLOUDFLARE_FLUX_STEPS  = 4

POLLINATIONS_BASE    = "https://image.pollinations.ai/prompt"
POLLINATIONS_MODEL   = os.getenv("POLLINATIONS_MODEL", "flux")
POLLINATIONS_TIMEOUT = 180

# Local Stable Diffusion model
SD_MODEL_ID         = "SG161222/Realistic_Vision_V5.1_noVAE"
CONTROLNET_MODEL_ID = "lllyasviel/control_v11p_sd15_lineart"

# Generation parameters (LCM-LoRA: 4 steps, CFG 1.5)
NUM_INFERENCE_STEPS = 4
GUIDANCE_SCALE      = 1.5
IMAGE_SIZE          = 512
NUM_IMAGES          = 4
MAX_NUM_IMAGES      = 4

# ── Face Matching ────────────────────────────────────────────
FACE_MODEL_DIR           = Path.home() / ".insightface" / "models" / "buffalo_l"
MATCH_STRUCTURAL_WEIGHT  = 0.40   # ArcFace structural score weight
MATCH_ATTRIBUTE_WEIGHT   = 0.60   # Attribute score weight
MIN_MATCH_SCORE          = 30.0
TOP_K_MATCHES            = 5

# ── Allowed upload extensions ────────────────────────────────
ALLOWED_IMAGE_EXTENSIONS = {"jpg", "jpeg", "png"}
ALLOWED_AUDIO_EXTENSIONS = {"wav", "mp3", "m4a", "ogg", "webm"}
```

9.2 Speech-to-Text Module

File: `modules/speech_to_text.py`

```
"""
Module 1: Speech-to-Text using Deepgram Nova-2 REST API
Converts multilingual witness audio into English text.
Zero RAM usage - all processing happens in the cloud.
"""

import mimetypes
from pathlib import Path
import requests
import config

DEEPGRAM_API_URL = "https://api.deepgram.com/v1/listen"
```

```
def transcribe_audio(audio_path: str, language: str = None) -> dict:
    """
    Transcribe a witness audio file using Deepgram Nova-2 REST API.

    Returns: {transcript, confidence, language}
    """
    api_key = config.DEEPGRAM_API_KEY

    if not api_key or api_key == "your_deepgram_api_key_here":
        return _fallback_no_key(audio_path)

    audio_file = Path(audio_path)
    if not audio_file.exists():
        return _fallback_no_key(audio_path)

    params = {
        "model":        config.DEEPGRAM_MODEL,
        "smart_format": "true",
        "punctuate":    "true",
    }
    if language:
        params["language"] = language
    else:
        params["detect_language"] = "true"
        params["language"] = config.DEEPGRAM_LANGUAGE

    mime_type, _ = mimetypes.guess_type(str(audio_file))
    if not mime_type:
        mime_type = "audio/wav"

    headers = {
        "Authorization": f"Token {api_key}",
        "Content-Type":  mime_type,
    }

    try:
        with open(audio_file, "rb") as f:
            response = requests.post(
                DEEPGRAM_API_URL,
                headers=headers,
                params=params,
                data=f.read(),
                timeout=60,
            )

        if response.status_code != 200:
            return {
                "transcript": f"[Deepgram error: {response.status_code}]",
                "confidence": 0.0,
                "language":   "error",
            }

        data        = response.json()
        channel     = data["results"]["channels"][0]
        alternative = channel["alternatives"][0]
        detected    = channel.get("detected_language", "unknown")
        confidence  = alternative.get("confidence", 0.0)
```

```
        return {
            "transcript": alternative["transcript"],
            "confidence": round(confidence * 100, 1),
            "language":   detected,
        }
    except requests.exceptions.ConnectionError:
        return _fallback_no_key(audio_path)
    except Exception as e:
        return {
            "transcript": f"[Error: {e}]",
            "confidence": 0.0,
            "language":   "error",
        }


def _fallback_no_key(audio_path: str) -> dict:
    """Fallback sample transcript when Deepgram key is not configured."""
    sample = (
        "The suspect was a male, around 30 to 35 years old. "
        "He had dark brown skin, short black hair, and brown eyes. "
        "He had a thick mustache and a small scar near his left eyebrow. "
        "He was of medium build."
    )
    return {
        "transcript": sample,
        "confidence": 0.0,
        "language":   "demo-mode",
    }
```

9.3 Attribute Parser

File: `modules/attribute_parser.py`

```
"""
Module 2: Forensic Attribute Parser
Extracts structured facial attributes from a witness description.
Every distinguishing mark (scar, mole, tattoo) is extracted with its
EXACT location on the face, because marks are the most identifying
features for forensic work.
"""

import re
from typing import Optional


# ── Attribute Vocabularies ──────────────────────────────

GENDER_KEYWORDS = {
    "female": [r"\bfemale\b", r"\bwoman\b", r"\bwomen\b", r"\bgirl\b",
               r"\blady\b", r"\bshe\b", r"\bher\b", r"\bhers\b", r"\bherself\b"],
    "male":   [r"\bmale\b", r"\bman\b", r"\bmen\b", r"\bboy\b",
               r"\bgentleman\b", r"\bhe\b", r"\bhis\b", r"\bhim\b", r"\bhimself\b"],
}

SKIN_TONES = ["very dark brown", "dark brown", "medium brown", "light brown",
              "wheatish", "olive", "tan", "medium", "fair", "light", "pale",
```

```
              "very fair", "dusky"]

HAIR_COLORS = ["jet black", "black", "dark brown", "brown", "light brown",
               "auburn", "red", "ginger", "blonde", "blond", "grey", "gray",
               "white", "silver", "salt and pepper"]

EYE_COLORS = ["black", "dark brown", "brown", "light brown", "hazel",
              "amber", "green", "blue-green", "blue", "grey", "gray"]

FACE_LOCATIONS = ["left cheek", "right cheek", "left eye", "right eye",
                  "below the left eye", "below the right eye",
                  "left eyebrow", "right eyebrow", "forehead", "chin",
                  "nose", "upper lip", "lower lip", "bridge of the nose",
                  "left temple", "right temple", "left jaw", "right jaw",
                  "corner of the mouth", "between the eyebrows"]

# POST-PROCESSING marks — rendered via OpenCV after AI generation.
MARK_TYPES = ["deep scar", "long scar", "small scar", "vertical scar",
              "horizontal scar", "surgical scar", "burn scar", "scar",
              "large mole", "small mole", "dark mole", "raised mole", "mole",
              "birthmark", "port wine stain", "wart", "bump"]

# PROMPT-SIDE accessories — rendered by the AI model inside the photo.
ACCESSORY_TYPES = ["bindi", "red bindi", "nose ring", "nose stud",
                   "gold nose ring", "earring", "hoop earring",
                   "stud earring", "nose piercing", "ear piercing",
                   "eyebrow piercing", "tattoo", "spectacles", "glasses",
                   "sunglasses", "chain", "necklace"]

# Religious / cultural attire. Handled separately from accessories so we
# can inject a strong prompt clause ("wearing X covering head and hair")
# and push ethnicity negatives that block the model's default drift.
RELIGIOUS_ATTIRE = ["black burka", "burka", "burqa", "niqab", "hijab",
                    "abaya", "chador", "headscarf", "head scarf", "dupatta",
                    "ghoonghat", "veil", "turban", "sikh turban", "pagri",
                    "dastar", "skullcap", "taqiyah", "kufi",
                    "yarmulke", "kippah"]


def parse_description(text: str) -> dict:
    """Extract structured facial attributes from a witness description."""
    text_lower = f" {text.lower().strip()} "

    attrs = {
        "gender":          _extract_gender(text_lower),
        "age":             _extract_age(text_lower),
        "ethnicity":       _extract_ethnicity(text_lower),
        "skin_tone":       _extract_adjacent(text_lower, SKIN_TONES,
                                             after_words=["skin", "complexion"]),
        "face_shape":      _extract_from_list(text_lower, FACE_SHAPES,
                                              context_words=["face", "shaped"]),
        "eyebrows":        _extract_from_list(text_lower, EYEBROW_TYPES,
                                              context_words=["eyebrow", "brow"]),
        "eye_color":       _extract_adjacent(text_lower, EYE_COLORS,
                                             after_words=["eye", "eyes"]),
        "nose":             _extract_adjacent(text_lower, NOSE_TYPES,
                                              after_words=["nose"]),
        "facial_hair":      _extract_from_list(text_lower, FACIAL_HAIR),
        "hair_color":       _extract_hair_color_detailed(text_lower),
```

```
        "hair_style":       _extract_adjacent(text_lower, HAIR_STYLES,
                                              after_words=["hair", "head"]),
        "build":            _extract_build(text_lower),
        "marks":            _extract_marks_precise(text_lower),
        "accessories":      _extract_accessories_with_sides(text_lower),
        "religious_attire": _extract_religious_attire(text_lower),
    }

    # Infer ethnicity from religious attire when witness did not give one.
    # "muslim woman in a burka" used to drop ethnicity, and the model's
    # dataset bias then produced African-looking faces. Attire is a
    # strong regional/cultural signal — use it as a fallback anchor.
    if not attrs["ethnicity"]:
        attire = attrs["religious_attire"]
        if any(a in ("burka", "burqa", "niqab", "hijab",
                     "abaya", "chador") for a in attire) or "muslim" in text_lower:
            attrs["ethnicity"] = "Muslim South Asian or Middle Eastern"
        elif any(a in ("turban", "sikh turban", "pagri", "dastar")
                 for a in attire):
            attrs["ethnicity"] = "North Indian Punjabi Sikh"
        elif any(a in ("dupatta", "ghoonghat") for a in attire):
            attrs["ethnicity"] = "South Asian Indian"

    return attrs


def _extract_religious_attire(text: str) -> list[str]:
    """Extract head/body religious attire keywords (longest match wins)."""
    found, matched_spans = [], []
    for item in sorted(RELIGIOUS_ATTIRE, key=len, reverse=True):
        idx = text.find(item)
        if idx == -1:
            continue
        end = idx + len(item)
        if any(idx < me and end > ms for ms, me in matched_spans):
            continue
        found.append(item)
        matched_spans.append((idx, end))
    return found


def _extract_marks_precise(text: str) -> list:
    """
    Extract distinguishing marks with PRECISE location. The closest
    location in the sentence wins, longer (more specific) locations
    break ties. This prevents cross-contamination when several marks
    share a sentence.
    """
    marks = []
    sentences = re.split(r'[.!?]+', text)
    for sentence in sentences:
        sentence = sentence.strip()
        if not sentence:
            continue

        for mark_type in MARK_TYPES:
            if mark_type not in sentence:
                continue
            idx = sentence.find(mark_type)
```

```
            # Two-pass: collect candidate locations (longest first),
            # then pick the closest to the mark.
            candidates, consumed = [], []
            for loc in sorted(FACE_LOCATIONS, key=len, reverse=True):
                start = 0
                while True:
                    li = sentence.find(loc, start)
                    if li == -1:
                        break
                    le = li + len(loc)
                    covered = any(li < ce and le > cs for cs, ce in consumed)
                    if not covered:
                        candidates.append((loc, li, le))
                        consumed.append((li, le))
                    start = li + 1

            best_loc, best_dist = None, float("inf")
            for loc, li, _ in candidates:
                d = abs(li - idx)
                if d < best_dist or (d == best_dist and best_loc
                                     and len(loc) > len(best_loc)):
                    best_loc, best_dist = loc, d

            marks.append({
                "type":        mark_type,
                "location":    best_loc,
                "descriptors": [],
                "raw":         sentence,
            })
    return marks


# ─── HiTS-style Prompt Builder ──────────────────────────────

def build_prompt(attrs: dict) -> tuple[str, str]:
    """
    Build a hierarchical HiTS-style prompt (Choi et al. 2025) for
    forensic face generation.

    Positive prompt structure:
      1. CUFS-format anchor sentence (race, gender, hair, skin, eyes)
      2. Phenotype reinforcement (ethnicity-specific features)
      3. Religious attire clause (hijab / burka / turban)
      4. Accessories (bindi, earrings, nose ring)
      5. Secondary features (face shape, nose, facial hair, build)

    Negative prompt is COLOR-BIASED — explicitly excludes competitors
    of the specified skin/hair/eye colors, fighting the SDXL/FLUX
    dataset bias toward an 'average' appearance.
    """
    gender    = attrs.get("gender")
    ethnicity = attrs.get("ethnicity") or ""
    skin      = attrs.get("skin_tone") or ""
    hair_c    = attrs.get("hair_color") or ""
    eye_c     = attrs.get("eye_color")  or ""

    # ── 1. CUFS anchor sentence ──
    intrinsic = _race_gender_phrase(attrs)
```

```
    parts = []
    if hair_c: parts.append(f"{hair_c} hair")
    if skin:   parts.append(f"{skin} skin")
    if eye_c:  parts.append(f"{eye_c} eyes")
    tail = (", ".join(parts[:-1]) + (", and " + parts[-1])
            if len(parts) > 1 else (parts[0] if parts else ""))
    cufs = f"A photo of a {intrinsic}" + (f" with {tail}" if tail else "") + "."

    # ── 2. Phenotype reinforcement ──
    phenotype = (_ETH_PHENOTYPE[ethnicity] + "."
                 if ethnicity in _ETH_PHENOTYPE else "")

    # ── 3. Religious / cultural attire — strongest clothing signal ──
    attire_phrases = []
    for item in attrs.get("religious_attire", []):
        phrase = _ATTIRE_PROMPT.get(item, f"wearing a {item}")
        if phrase not in attire_phrases:
            attire_phrases.append(phrase)
    attire_line = ", ".join(attire_phrases) + "." if attire_phrases else ""

    # ── 4. Accessories line ──
    accessories  = attrs.get("accessories", []) or []
    accessory_line = ("Wearing: " + ", ".join(accessories) + "."
                      if accessories else "")

    # ── 5. Secondary features ──
    secondary = [f"{attrs[k]} {k.replace('_', ' ')}"
                 for k in ("face_shape", "eyebrows", "nose", "lips",
                           "chin", "facial_hair", "build")
                 if attrs.get(k)]
    secondary_line = "Features: " + ", ".join(secondary) + "." if secondary else ""

    positive = " ".join(filter(None, [
        cufs, phenotype, attire_line, accessory_line, secondary_line,
    ]))

    # ── Color-biased negative prompt ──
    negatives = []
    if skin.lower() in _SKIN_COMPETITORS:
        negatives.extend(_SKIN_COMPETITORS[skin.lower()])
    if hair_c.lower() in _HAIR_COMPETITORS:
        negatives.extend(_HAIR_COMPETITORS[hair_c.lower()])
    if eye_c.lower() in _EYE_COMPETITORS:
        negatives.extend(_EYE_COMPETITORS[eye_c.lower()])
    if ethnicity in _ETH_NEGATIVE:
        negatives.extend(_ETH_NEGATIVE[ethnicity])

    base_negative = (
        "cartoon, anime, illustration, painting, drawing, sketch, 3D "
        "render, CGI, blurry, deformed, watermark, text, multiple "
        "people, side view, profile, smiling, open mouth, hat, mask"
    )
    if negatives:
        seen = set()
        unique = [c for c in negatives if not (c in seen or seen.add(c))]
        negative = ", ".join(unique) + ", " + base_negative
    else:
        negative = base_negative
```

```
    return positive, negative
```

9.4 Face Generator

File: `modules/face_generator.py`

```
"""
Module 3: Forensic Face Generation
Two modes, selected via config.GENERATION_MODE:
  "local" → SD 1.5 Realistic Vision + LCM-LoRA locally, includes
            ControlNet sketch-to-photo.
  "api"   → FLUX.1-schnell via Cloudflare Workers AI or Pollinations.
"""
import time
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path
from PIL import Image
import config

_sd_pipe         = None
_controlnet_pipe = None


# ═══════════════════════════════════════════════════════════
# SFW guardrails — applied to every generation path.
# ═══════════════════════════════════════════════════════════

_CLOTHING_HINT = "wearing gray shirt, clothed."

_SFW_NEGATIVE = (
    "nude, naked, nudity, nsfw, topless, shirtless, bare chest, "
    "bare shoulders, exposed skin below neck, underwear, bikini, "
    "cleavage, sexual, suggestive, "
)

_STYLE_FILL = [
    "front-facing mugshot,", "neutral expression,", "gray background,",
    "realistic photo,", "sharp focus,", "detailed skin texture,",
    "85mm lens,", "DSLR,", "studio lighting,", "direct eye contact,",
    "one person,", "high resolution.",
]


def _apply_safety(positive: str, negative: str) -> tuple[str, str]:
    """Build a 75-CLIP-token prompt: identity + clothing + style fillers."""
    pos = positive.strip()
    if "clothed" not in pos.lower():
        pos = f"{pos} {_CLOTHING_HINT}"
    for phrase in _STYLE_FILL:
        candidate = f"{pos} {phrase}"
        if _count_tokens(candidate) <= 75:
            pos = candidate
        else:
            break
    neg = (negative or "").strip()
    if "nude" not in neg.lower():
```

```
        neg = _SFW_NEGATIVE + neg
    return pos, neg


# ═══════════════════════════════════════════════════════════════
# Pencil Sketch — one of the N variants is rendered as a
# forensic-artist pencil drawing via the color-dodge technique.
# ═══════════════════════════════════════════════════════════════

def _to_pencil_sketch(img: Image.Image) -> Image.Image:
    """
    Convert a realistic face photo into a pencil-drawing style using
    the classic color-dodge technique (gray + inverted-blurred-gray
    dodge).
    """
    import numpy as np
    import cv2

    arr  = np.array(img.convert("RGB"))
    gray = cv2.cvtColor(arr, cv2.COLOR_RGB2GRAY)
    inv  = 255 - gray
    blur = cv2.GaussianBlur(inv, (21, 21), 0, 0)

    denom = 255 - blur
    denom[denom == 0] = 1
    dodge = cv2.divide(gray, denom, scale=256.0)
    dodge = np.clip(dodge, 0, 255).astype(np.uint8)

    # Slight contrast lift so lines read as pencil strokes.
    dodge = cv2.addWeighted(dodge, 1.15, dodge, 0, -15)
    dodge = np.clip(dodge, 0, 255).astype(np.uint8)
    rgb   = cv2.cvtColor(dodge, cv2.COLOR_GRAY2RGB)
    return Image.fromarray(rgb)


# ═══════════════════════════════════════════════════════════════
# API MODE: FLUX.1-schnell via Cloudflare or Pollinations.
# FLUX runs at CFG 0 and ignores negatives — all ethnicity/quality
# steering lives in the positive prompt.
# ═══════════════════════════════════════════════════════════════

def generate_from_api(
    positive_prompt: str,
    negative_prompt: str,
    num_images: int = 4,
) -> list[Image.Image]:
    """N candidates via the selected free API provider, in parallel."""
    base_prompt = positive_prompt.strip()
    if "clothed" not in base_prompt.lower():
        base_prompt = f"{base_prompt} {_CLOTHING_HINT}"
    base_prompt = (
        f"{base_prompt} front-facing forensic mugshot, neutral "
        f"expression, even gray studio background, realistic photograph, "
        f"sharp focus, detailed skin texture, direct eye contact, "
        f"one person only, high resolution, photorealistic."
    )

    base_seed = 7331
    lighting_hints = [
```

```
        "soft even studio lighting", "clinical flash lighting",
        "warm tungsten lighting", "neutral daylight balanced lighting",
    ]

    n = min(num_images, config.MAX_NUM_IMAGES)
    use_cf = config.USE_CLOUDFLARE
    # Pollinations free tier allows only 1 queued request per IP.
    max_workers = n if use_cf else 1

    def _one(i: int) -> tuple[int, Image.Image]:
        variant = f"{base_prompt} {lighting_hints[i % len(lighting_hints)]}."
        if use_cf:
            img = _cloudflare_flux_request(variant, seed=base_seed + i)
        else:
            img = _pollinations_request(variant, seed=base_seed + i)
        img = img.resize((config.IMAGE_SIZE, config.IMAGE_SIZE), Image.LANCZOS)
        save_path = config.GENERATED_DIR / f"generated_{i + 1}.png"
        img.save(save_path)
        return i, img

    results: list[Image.Image | None] = [None] * n
    with ThreadPoolExecutor(max_workers=max_workers) as pool:
        for i, img in pool.map(_one, range(n)):
            results[i] = img
    return [img for img in results if img is not None]


# ═══════════════════════════════════════════════════════════
# Dispatcher — Stage 1: AI generate, Stage 2: render marks,
# Stage 3: overwrite last slot with pencil-sketch variant.
# ═══════════════════════════════════════════════════════════

def generate(
    positive_prompt: str,
    negative_prompt: str,
    num_images: int = 4,
    sketch_image: Image.Image = None,
    marks: list = None,
) -> list[Image.Image]:
    """Full two-stage pipeline with an optional sketch variant."""
    mode = (config.GENERATION_MODE or "local").lower()
    num_images = min(num_images, config.MAX_NUM_IMAGES)

    # ── Stage 1: AI generation ──
    if mode == "local":
        if sketch_image is not None:
            base_images = generate_from_sketch(
                sketch_image, positive_prompt, negative_prompt, num_images)
        else:
            base_images = generate_from_text(
                positive_prompt, negative_prompt, num_images)
    elif mode == "api":
        base_images = generate_from_api(
            positive_prompt, negative_prompt, num_images)
    else:
        raise RuntimeError(f"Unknown GENERATION_MODE={mode!r}")

    # ── Stage 2: render scars/moles at anatomical landmarks ──
    if marks:
```

```
        from modules.face_enhancer import add_marks_to_face
        enhanced = []
        for i, img in enumerate(base_images):
            e = add_marks_to_face(img, marks)
            enhanced.append(e)
            (config.GENERATED_DIR / f"generated_{i + 1}.png").write_bytes(
                b"")  # placeholder — e.save used in real code
            e.save(config.GENERATED_DIR / f"generated_{i + 1}.png")
        base_images = enhanced

    # ── Stage 3: render one pencil-sketch variant ──
    # When the officer asks for ≥2 variants, the last slot is replaced
    # with a forensic-artist pencil drawing of variant #1. A single-
    # image run stays as a photo so ArcFace matching keeps its input.
    if len(base_images) >= 2:
        sketch_idx = len(base_images) - 1
        sketch_img = _to_pencil_sketch(base_images[0])
        sketch_path = config.GENERATED_DIR / f"generated_{sketch_idx + 1}.png"
        sketch_img.save(sketch_path)
        base_images[sketch_idx] = sketch_img

    return base_images
```

9.5 Face Enhancer (Mark Rendering)

File: `modules/face_enhancer.py`

```
"""
Module 5: Forensic Mark Renderer
Renders scars, moles, and distinguishing marks onto generated faces
at PRECISE locations using face landmarks.

Uses OpenCV compositing — deterministic, realistic, no model downloads.
"""
import random
from typing import Optional
import cv2
import numpy as np
from PIL import Image
from modules.onnx_face import OnnxFaceProcessor

_processor: Optional[OnnxFaceProcessor] = None


def _get_processor() -> OnnxFaceProcessor:
    global _processor
    if _processor is None:
        _processor = OnnxFaceProcessor()
        _processor.load_detection()
    return _processor


def _compute_face_regions(bbox, lmks) -> dict[str, tuple[int, int]]:
    """
    Map verbal location descriptions → pixel coordinates using the
    5-point landmarks from RetinaFace.
```

```
    IMPORTANT: RetinaFace returns landmarks in viewer perspective,
    but witness descriptions are always in SUBJECT perspective.
    Rendering a mole on the wrong side of the face is a serious
    forensic error — so we bind SUBJECT-pov variables here once.
    """
    subj_right_eye   = lmks[0]   # at viewer's left
    subj_left_eye    = lmks[1]   # at viewer's right
    nose             = lmks[2]
    subj_right_mouth = lmks[3]
    subj_left_mouth  = lmks[4]

    eye_dist = np.linalg.norm(subj_left_eye - subj_right_eye)
    unit     = eye_dist / 4

    return {
        "left eye":         _pt(subj_left_eye),
        "right eye":        _pt(subj_right_eye),
        "below left eye":   _pt(subj_left_eye  + [0, unit * 1.2]),
        "below right eye":  _pt(subj_right_eye + [0, unit * 1.2]),
        "above left eye":   _pt(subj_left_eye  + [0, -unit * 1.0]),
        "above right eye":  _pt(subj_right_eye + [0, -unit * 1.0]),
        "left cheek":       _pt(subj_left_mouth  + [0, -unit * 0.8]),
        "right cheek":      _pt(subj_right_mouth + [0, -unit * 0.8]),
        "nose":             _pt(nose),
        "chin":             _pt((subj_right_mouth + subj_left_mouth) / 2
                                + [0, unit * 2.5]),
        "forehead":         _pt((subj_left_eye + subj_right_eye) / 2
                                + [0, -unit * 3.0]),
    }


def render_mole(image, center, size="small", descriptors=None, eye_dist=100.0):
    """
    Render a realistic mole using skin colour sampling: a darker-than-
    skin ellipse with soft Gaussian edges and a subtle 3D highlight.
    """
    img = image.copy()
    h, w = img.shape[:2]
    x, y = max(5, min(w - 5, center[0])), max(5, min(h - 5, center[1]))

    scale = eye_dist / 100.0
    size_map = {"tiny": int(2 * scale), "small": int(4 * scale),
                "medium": int(6 * scale), "large": int(9 * scale)}
    radius = size_map.get(size, size_map["small"])

    skin_color  = _sample_skin_color(img, (x, y))
    darkness    = -50 if descriptors and "dark" in descriptors else -35
    mole_color  = np.clip(skin_color.astype(int) + darkness, 10, 180)

    mask  = np.zeros((h, w), dtype=np.float32)
    axes  = (radius, max(2, int(radius * 0.85)))
    angle = random.randint(0, 180)
    cv2.ellipse(mask, (x, y), axes, angle, 0, 360, 1.0, -1)

    k = max(3, radius * 2 + 1)
    if k % 2 == 0: k += 1
    mask = cv2.GaussianBlur(mask, (k, k), 0)

    mole_layer = np.full_like(img, mole_color, dtype=np.uint8)
```

```
    mask_3ch   = np.stack([mask] * 3, axis=-1)
    img = (img.astype(np.float32) * (1 - mask_3ch * 0.92) +
           mole_layer.astype(np.float32) * mask_3ch * 0.92).astype(np.uint8)
    return img


def add_marks_to_face(image, marks: list[dict]) -> Image.Image:
    """
    Public entrypoint — detect landmarks, resolve each mark's verbal
    location to pixel coordinates, and call the appropriate renderer.
    """
    if isinstance(image, str):
        img = cv2.imread(image)
    elif isinstance(image, Image.Image):
        img = cv2.cvtColor(np.array(image.convert("RGB")), cv2.COLOR_RGB2BGR)
    else:
        img = image.copy()

    if not marks:
        return Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

    faces = _get_processor().detect_faces(img, score_thresh=0.3)
    if not faces:
        return Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

    face      = faces[0]
    bbox      = face["bbox"]
    landmarks = face["landmarks_5"]
    eye_dist  = float(np.linalg.norm(landmarks[1] - landmarks[0]))

    for mark in marks:
        mark_type    = mark.get("type", "")
        location_str = mark.get("location", "")
        descriptors  = mark.get("descriptors", [])

        center = resolve_location(location_str, bbox, landmarks)
        if center is None:
            continue

        renderer = _MARK_RENDERERS.get(mark_type)
        if renderer is None:
            for key, func in _MARK_RENDERERS.items():
                if key in mark_type or mark_type in key:
                    renderer = func
                    break
        if renderer is None:
            continue

        img = renderer(img, center,
                       size=descriptors[0] if descriptors else "medium",
                       descriptors=descriptors, eye_dist=eye_dist)

    return Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

9.6 ONNX Face Processor

File: `modules/onnx_face.py`

```
"""
Pure ONNX Face Processor — Zero insightface dependency.
Uses RetinaFace (det_10g.onnx) for detection + 5-point landmarks,
and ArcFace (w600k_r50.onnx) for 512-d embeddings.
"""
from pathlib import Path
from typing import Optional
import cv2
import numpy as np
from PIL import Image

_DEFAULT_MODEL_DIR = Path.home() / ".insightface" / "models" / "buffalo_l"
_DET_MODEL_NAME    = "det_10g.onnx"
_REC_MODEL_NAME    = "w600k_r50.onnx"
_FPN_STRIDES       = [8, 16, 32]
_NUM_ANCHORS       = 2
_DET_SIZE          = (640, 640)

# Standard ArcFace alignment reference — 5 points in 112×112 crop.
_ARCFACE_DST = np.array([
    [38.2946, 51.6963], [73.5318, 51.5014], [56.0252, 71.7366],
    [41.5493, 92.3655], [70.7299, 92.2041],
], dtype=np.float32)


class OnnxFaceProcessor:
    """
    Standalone face detection + recognition using ONNX models directly.
    Drop-in replacement for insightface FaceAnalysis.
    """

    def __init__(self, model_dir=None):
        self.model_dir     = Path(model_dir or _DEFAULT_MODEL_DIR)
        self._det_session  = None
        self._rec_session  = None
        self._anchor_cache = {}

    def _get_providers(self):
        """Prefer CUDA if available, fall back to CPU."""
        import onnxruntime as ort
        available = ort.get_available_providers()
        providers = []
        if "CUDAExecutionProvider" in available:
            providers.append("CUDAExecutionProvider")
        providers.append("CPUExecutionProvider")
        return providers

    def load_detection(self):
        import onnxruntime as ort
        if self._det_session is not None:
            return
        self._det_session = ort.InferenceSession(
            str(self.model_dir / _DET_MODEL_NAME),
            providers=self._get_providers())

    def load_recognition(self):
        import onnxruntime as ort
        if self._rec_session is not None:
```

```
            return
        self._rec_session = ort.InferenceSession(
            str(self.model_dir / _REC_MODEL_NAME),
            providers=self._get_providers())

    def detect_faces(self, img_bgr, score_thresh=0.5, nms_thresh=0.4):
        """Detect faces. Returns list of dicts: {bbox, score, landmarks_5}."""
        self.load_detection()

        blob, scale, _ = self._preprocess_det(img_bgr)
        det_h, det_w   = _DET_SIZE

        input_name = self._det_session.get_inputs()[0].name
        outputs    = self._det_session.run(None, {input_name: blob})

        # Parse outputs: 9 arrays, 3 per FPN level (scores, boxes, lmks)
        anchors = self._generate_anchors(det_h, det_w)
        all_scores, all_boxes, all_landmarks = [], [], []

        for level_idx, (centers, stride) in enumerate(anchors):
            scores = outputs[level_idx]
            boxes  = outputs[level_idx + 3]
            lmks   = outputs[level_idx + 6]
            mask   = scores[:, 0] > score_thresh
            if not mask.any():
                continue

            fs, fb, fl, fc = scores[mask, 0], boxes[mask], lmks[mask], centers[mask]

            # Decode: distance-from-anchor boxes, offset landmarks.
            x1 = (fc[:, 0] - fb[:, 0]) * stride
            y1 = (fc[:, 1] - fb[:, 1]) * stride
            x2 = (fc[:, 0] + fb[:, 2]) * stride
            y2 = (fc[:, 1] + fb[:, 3]) * stride
            decoded_boxes = np.stack([x1, y1, x2, y2], axis=1)

            decoded_lmks = np.zeros_like(fl)
            for k in range(5):
                decoded_lmks[:, k * 2]     = (fc[:, 0] + fl[:, k * 2])     * stride
                decoded_lmks[:, k * 2 + 1] = (fc[:, 1] + fl[:, k * 2 + 1]) * stride

            all_scores.append(fs)
            all_boxes.append(decoded_boxes)
            all_landmarks.append(decoded_lmks)

        if not all_scores:
            return []
        scores    = np.concatenate(all_scores)
        boxes     = np.concatenate(all_boxes)
        landmarks = np.concatenate(all_landmarks)
        keep      = self._nms(boxes, scores, nms_thresh)

        results = []
        for i in keep:
            bbox = boxes[i] / scale
            lmk  = landmarks[i].reshape(5, 2) / scale
            results.append({
                "bbox":        bbox.astype(np.float32),
                "score":       float(scores[i]),
```

```
                "landmarks_5": lmk.astype(np.float32),
            })
        results.sort(key=lambda f: (f["bbox"][2] - f["bbox"][0])
                                 * (f["bbox"][3] - f["bbox"][1]),
                     reverse=True)
        return results

    @staticmethod
    def align_face(img_bgr: np.ndarray, landmarks_5: np.ndarray) -> np.ndarray:
        """Warp face to 112×112 ArcFace alignment using similarity transform."""
        tform = _estimate_similarity_transform(
            landmarks_5.astype(np.float32), _ARCFACE_DST.copy())
        return cv2.warpAffine(img_bgr, tform, (112, 112), borderValue=0)

    def get_embedding(self, img_bgr, landmarks_5):
        """Extract 512-d ArcFace embedding from a face."""
        self.load_recognition()
        aligned = self.align_face(img_bgr, landmarks_5)
        blob    = (aligned.astype(np.float32) - 127.5) / 127.5
        blob    = blob.transpose(2, 0, 1)[np.newaxis]
        input_name = self._rec_session.get_inputs()[0].name
        return self._rec_session.run(None, {input_name: blob})[0][0]

    def get_face_embedding(self, image_input) -> Optional[np.ndarray]:
        """High-level API: detect largest face, return 512-d embedding."""
        if isinstance(image_input, (str, Path)):
            img_bgr = cv2.imread(str(image_input))
        elif isinstance(image_input, Image.Image):
            img_bgr = cv2.cvtColor(np.array(image_input.convert("RGB")),
                                   cv2.COLOR_RGB2BGR)
        else:
            img_bgr = image_input
        if img_bgr is None:
            return None

        faces = self.detect_faces(img_bgr, score_thresh=0.3)
        if not faces:
            return None
        return self.get_embedding(img_bgr, faces[0]["landmarks_5"])
```

9.7 Face Matcher (Two-Layer)

File: `modules/face_matcher.py`

```
"""
Module 4: Two-Layer Face Matching System

Layer 1 — ArcFace Structural Matching:
  Compares 512-d face embeddings via cosine similarity.
Layer 2 — Attribute Matching:
  Compares witness-described attributes against DB metadata.

Final Score = 0.40 * structural + 0.60 * attribute
"""
import json, pickle, time
from pathlib import Path
import numpy as np
```

```
import cv2
from PIL import Image
import config
from modules.onnx_face import OnnxFaceProcessor, ensure_models


# Distinguishing marks are weighted HIGHEST because they are the
# single most identifying feature in forensic contexts.
ATTR_WEIGHTS = {
    "gender":      15, "age": 5, "skin_tone":   10,
    "hair_color":   8, "hair_style":  5, "eye_color":    5,
    "facial_hair":  8, "face_shape":  3, "nose":         3,
    "build":        2, "marks":      30,
}
STRUCTURAL_WEIGHT = 0.40
ATTRIBUTE_WEIGHT  = 0.60


class FaceMatcher:
    """Two-layer face matching engine."""

    def __init__(self):
        self.processor   = None
        self.db_entries  = []
        self._db_cache_path = config.MODELS_DIR / "db_embeddings_v4.pkl"
        self._db_matrix  = None   # (N, 512) stacked normalized embeddings

    def load_model(self):
        if self.processor is not None:
            return
        try:
            ensure_models()
        except Exception:
            pass
        self.processor = OnnxFaceProcessor()
        self.processor.load()

    def get_embedding(self, image_input):
        self.load_model()
        return self.processor.get_face_embedding(image_input)

    def build_database(self, db_dir=None):
        """
        Build database from face images + attribute JSON metadata.
        For each image, loads attributes from:
          1. Sidecar .json (e.g. abc123.json next to abc123.jpg)
          2. SQLite criminals table (attributes_json column)
        """
        db_dir      = Path(db_dir or config.CRIMINAL_DB_DIR)
        image_files = sorted(
            list(db_dir.glob("*.jpg")) + list(db_dir.glob("*.jpeg"))
            + list(db_dir.glob("*.png")))
        if not image_files:
            return

        self.load_model()
        self.db_entries = []
        db_attrs = self._load_db_attributes()
```

```
        for img_path in image_files:
            stem      = img_path.stem
            embedding = self.get_embedding(str(img_path))
            if embedding is None:
                continue

            # L2-normalize once at build time so match() can skip it.
            norm = np.linalg.norm(embedding)
            if norm > 0:
                embedding = embedding / norm

            meta_path  = img_path.with_suffix(".json")
            attributes = {}
            if meta_path.exists():
                with open(meta_path) as f:
                    attributes = json.load(f)
            elif stem in db_attrs:
                attributes = db_attrs[stem]["attributes"]

            self.db_entries.append({
                "name":       db_attrs[stem]["name"] if stem in db_attrs else stem,
                "embedding":  embedding.astype(np.float32),
                "image_path": str(img_path),
                "attributes": attributes,
            })

        self._rebuild_matrix()
        with open(self._db_cache_path, "wb") as f:
            pickle.dump(self.db_entries, f)

    def match(self, query_image, query_attributes=None, top_k=None):
        """
        Two-layer match with gender as a HARD filter.

        ArcFace embeddings don't strongly separate male vs female — a
        male DB face can score 60–70 % structurally against a female
        query. Scoring gender as an attribute only moves that a few
        points, which isn't enough. So we use gender as a GATE: if the
        witness specified a gender AND the DB entry has an explicit
        opposite gender, that entry is excluded entirely.
        """
        top_k = top_k or config.TOP_K_MATCHES
        if not self.db_entries:
            self.load_database()
        if not self.db_entries:
            return []

        # ── Hard gender filter ──
        query_gender = None
        if query_attributes:
            g = query_attributes.get("gender")
            if g:
                query_gender = str(g).strip().lower()

        eligible = []
        for i, entry in enumerate(self.db_entries):
            if query_gender:
                db_g = entry.get("attributes", {}).get("gender")
                if db_g and str(db_g).strip().lower() != query_gender:
```

```
                continue
            eligible.append(i)
        if not eligible:
            return []

        # ── Layer 1: ArcFace structural similarity ──
        query_emb = self.get_embedding(query_image)
        if query_emb is None:
            structural = {i: 0.0 for i in eligible}
        else:
            q_norm = query_emb / (np.linalg.norm(query_emb) + 1e-8)
            if self._db_matrix is None:
                self._rebuild_matrix()
            m    = self._db_matrix[eligible]
            sims = np.clip(m @ q_norm.astype(np.float32), 0.0, 1.0) * 100.0
            structural = {i: float(sims[k]) for k, i in enumerate(eligible)}

        # ── Layer 2: Attribute scoring + final combination ──
        results = []
        for i in eligible:
            entry = self.db_entries[i]
            s     = structural[i]
            a, matched, mis = 0, [], []
            if query_attributes and entry["attributes"]:
                a, matched, mis = self._compare_attributes(
                    query_attributes, entry["attributes"])
                final = STRUCTURAL_WEIGHT * s + ATTRIBUTE_WEIGHT * a
            else:
                final = s

            results.append({
                "name":             entry["name"],
                "final_score":      round(final, 1),
                "structural_score": round(s, 1),
                "attribute_score":  round(a, 1),
                "matched_attrs":    matched,
                "mismatched_attrs": mis,
                "image_path":       entry["image_path"],
            })

        results.sort(key=lambda x: x["final_score"], reverse=True)
        return results[:top_k]

    def _compare_attributes(self, query, db_attrs):
        """Attribute scoring — marks weighted highest, age ±7y tolerance."""
        total_w, earned = 0, 0
        matched, mismatched = [], []

        for attr_key, weight in ATTR_WEIGHTS.items():
            if attr_key == "marks":
                continue
            q_val = query.get(attr_key)
            d_val = db_attrs.get(attr_key)
            if q_val is None:
                continue
            total_w += weight

            if d_val is None:
                earned += weight * 0.5
```

```
                continue
            q_str, d_str = str(q_val).lower(), str(d_val).lower()

            if attr_key == "age":
                diff = abs(int(q_val) - int(d_val))
                if diff <= 3:
                    earned += weight
                    matched.append(f"Age ~{d_val}")
                elif diff <= 7:
                    earned += weight * 0.6
                else:
                    mismatched.append(f"Age: {q_val} vs {d_val}")
            elif attr_key == "gender":
                if q_str == d_str:
                    earned += weight
                    matched.append(f"Gender: {d_str}")
                else:
                    mismatched.append(f"Gender mismatch")
            else:
                if q_str == d_str or q_str in d_str or d_str in q_str:
                    earned += weight
                    matched.append(f"{attr_key}: {d_str}")
                else:
                    overlap = set(q_str.split()) & set(d_str.split())
                    if overlap:
                        earned += weight * 0.5

        # ── Marks weighted 30 points — highest in forensic context ──
        q_marks, d_marks = query.get("marks", []), db_attrs.get("marks", [])
        if q_marks:
            total_w += ATTR_WEIGHTS["marks"]
            if d_marks:
                matched_count = 0
                for qm in q_marks:
                    q_type = qm.get("type", "")
                    q_loc  = qm.get("location", "")
                    for dm in d_marks:
                        d_type = dm.get("type", "")
                        d_loc  = dm.get("location", "")
                        if (q_type in d_type or d_type in q_type) and \
                           (q_loc == d_loc or (q_loc and q_loc in d_loc)):
                            matched_count += 1
                            matched.append(f"Mark: {q_type} @ {q_loc}")
                            break
                if matched_count:
                    earned += ATTR_WEIGHTS["marks"] * (matched_count / len(q_marks))

        score = (earned / total_w) * 100 if total_w else 0
        return round(score, 1), matched, mismatched
```

9.8 Flask Extensions

File: `extensions.py`

```
"""
Flask extensions and SQLite database layer.
All extensions initialised here to avoid circular imports.
```

```
"""
import sqlite3
from pathlib import Path
from flask_bcrypt import Bcrypt
from flask_login import LoginManager

bcrypt        = Bcrypt()
login_manager = LoginManager()
login_manager.login_view             = "auth.login"
login_manager.login_message          = "Please log in to access this page."
login_manager.login_message_category = "warning"


def get_db():
    """Return a fresh SQLite connection. Caller must close it."""
    import config
    conn = sqlite3.connect(str(config.DATABASE_PATH), timeout=30)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    conn.execute("PRAGMA journal_mode = WAL")
    conn.execute("PRAGMA busy_timeout = 30000")
    return conn


def init_db():
    """Create all tables if they don't exist yet."""
    schema_path = Path(__file__).parent / "database" / "schema.sql"
    conn = get_db()
    with open(schema_path, "r") as f:
        conn.executescript(f.read())
    conn.commit()
    conn.close()
    print("[DB] Schema initialised.")
```

9.9 Database Schema

File: `database/schema.sql`

```
-- Forensic Combined System — SQLite Schema

-- ── Users (all roles in one table) ──────────────────────────
CREATE TABLE IF NOT EXISTS users (
    id                   INTEGER PRIMARY KEY AUTOINCREMENT,
    username             TEXT NOT NULL UNIQUE,
    password_hash        TEXT NOT NULL,
    role                 TEXT NOT NULL CHECK(role IN ('superadmin','admin','police')),
    full_name            TEXT NOT NULL,
    email                TEXT UNIQUE,
    mobile               TEXT,
    police_id            TEXT UNIQUE,
    station_name         TEXT,
    location             TEXT,
    rank                 TEXT,
    is_active            INTEGER NOT NULL DEFAULT 1,
    force_password_change INTEGER NOT NULL DEFAULT 0,
    created_by           INTEGER REFERENCES users(id),
    created_at           TEXT DEFAULT (datetime('now')),
```

```
    last_login            TEXT
);

-- ── Criminal Records ──────────────────────────────────
CREATE TABLE IF NOT EXISTS criminals (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    criminal_code       TEXT UNIQUE,
    name                TEXT NOT NULL,
    age                 INTEGER,
    gender              TEXT,
    skin_tone           TEXT,
    crime_type          TEXT,
    crime_year          INTEGER,
    no_of_crimes        INTEGER DEFAULT 0,
    last_known_location TEXT,
    status              TEXT DEFAULT 'wanted'
                        CHECK(status IN ('wanted','arrested','released','deceased')),
    description         TEXT,
    face_image_path     TEXT,
    attributes_json     TEXT,
    added_by            INTEGER REFERENCES users(id),
    added_at            TEXT DEFAULT (datetime('now')),
    updated_at          TEXT
);

-- ── Investigation Cases ───────────────────────────────
CREATE TABLE IF NOT EXISTS cases (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    case_number       TEXT UNIQUE,
    officer_id        INTEGER NOT NULL REFERENCES users(id),
    incident_date     TEXT,
    incident_location TEXT,
    status            TEXT DEFAULT 'open'
                      CHECK(status IN ('open','closed','pending')),
    created_at        TEXT DEFAULT (datetime('now')),
    updated_at        TEXT
);

-- ── Investigation Pipeline Runs ───────────────────────
CREATE TABLE IF NOT EXISTS investigation_runs (
    id                    INTEGER PRIMARY KEY AUTOINCREMENT,
    case_id               INTEGER REFERENCES cases(id) ON DELETE CASCADE,
    officer_id            INTEGER NOT NULL REFERENCES users(id),

    -- Step 1: STT
    audio_file_path       TEXT,
    transcript            TEXT,
    stt_language          TEXT,
    stt_confidence        REAL,

    -- Step 2: Attribute parser
    description_text      TEXT,
    parsed_attributes     TEXT,
    positive_prompt       TEXT,
    negative_prompt       TEXT,

    -- Step 3: Face generation
    generated_image_paths TEXT,
    generation_mode       TEXT,
```

```
    had_sketch_input      INTEGER DEFAULT 0,

    -- Step 4: Matching
    selected_image_path   TEXT,
    match_results         TEXT,
    top_match_name        TEXT,
    top_match_score       REAL,

    run_at                TEXT DEFAULT (datetime('now'))
);

-- ── Audit Log ──────────────────────────────────────────
CREATE TABLE IF NOT EXISTS audit_log (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER REFERENCES users(id),
    action      TEXT NOT NULL,
    target_type TEXT,
    target_id   INTEGER,
    details     TEXT,
    ip_address  TEXT,
    logged_at   TEXT DEFAULT (datetime('now'))
);
```

9.10 Flask Application

File: `app.py`

```
"""
Forensic Combined System — Main Flask Application
Combines System A (AI pipeline) + System B (Flask / role-based UI).
"""
import json, threading, traceback, uuid
from datetime import timedelta, datetime
from pathlib import Path
from functools import wraps

from flask import (Flask, render_template, request, redirect, url_for,
                   session, flash, jsonify, send_from_directory, abort)
from flask_login import (login_user, logout_user, login_required,
                         current_user, UserMixin)
from werkzeug.utils import secure_filename

import config
from extensions import bcrypt, login_manager, init_db, get_db

from modules.speech_to_text   import transcribe_audio
from      modules.attribute_parser      import      parse_description,      build_prompt,
format_attributes_display
from modules.face_generator   import generate
from modules.face_matcher     import FaceMatcher


# ═══════════════════════════════════════════════════════
# App setup + Flask-Login user model
# ═══════════════════════════════════════════════════════

app = Flask(__name__)
```

```
app.secret_key                   = config.SECRET_KEY
app.permanent_session_lifetime   = timedelta(hours=config.SESSION_LIFETIME_HOURS)
app.config["MAX_CONTENT_LENGTH"] = config.MAX_UPLOAD_MB * 1024 * 1024

bcrypt.init_app(app)
login_manager.init_app(app)

# Singleton face matcher — loaded once and reused across requests.
_matcher = None

def get_matcher():
    global _matcher
    if _matcher is None:
        _matcher = FaceMatcher()
        _matcher.load_database()
    return _matcher


class User(UserMixin):
    def __init__(self, row):
        self.id         = row["id"]
        self.username   = row["username"]
        self.role       = row["role"]
        self.full_name  = row["full_name"]
        self._active    = bool(row["is_active"])
        self.force_pw   = bool(row["force_password_change"])

    @property
    def is_active(self):
        return self._active

    def get_id(self):
        return str(self.id)


@login_manager.user_loader
def load_user(user_id):
    conn = get_db()
    row = conn.execute("SELECT * FROM users WHERE id=?", (user_id,)).fetchone()
    conn.close()
    return User(row) if row else None


def role_required(*roles):
    def decorator(f):
        @wraps(f)
        def wrapped(*args, **kwargs):
            if not current_user.is_authenticated:
                return redirect(url_for("login"))
            if current_user.role not in roles:
                abort(403)
            if current_user.force_pw and request.endpoint != "change_password":
                return redirect(url_for("change_password"))
            return f(*args, **kwargs)
        return wrapped
    return decorator


# ═══════════════════════════════════════════════════════════
```

```
# Step 2 — parse witness text into attributes + prompts
# ═══════════════════════════════════════════════════════════

@app.route("/api/parse", methods=["POST"])
@login_required
@role_required("police")
def api_parse():
    data = request.get_json()
    text = data.get("text", "").strip()
    if not text:
        return jsonify({"error": "No description text"}), 400
    try:
        attrs = parse_description(text)
        positive, negative = build_prompt(attrs)
        return jsonify({
            "attributes":      attrs,
            "display":         format_attributes_display(attrs),
            "positive_prompt": positive,
            "negative_prompt": negative,
            "marks":           attrs.get("marks", []),
            "accessories":     attrs.get("accessories", []),
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 500


# ═══════════════════════════════════════════════════════════
# Step 3 — background face generation + polling endpoint
# ═══════════════════════════════════════════════════════════

_gen_status = {"running": False, "done": False, "error": None,
               "num": 0, "mode": "text"}
_gen_lock   = threading.Lock()


def _run_generation_job(positive, negative, num, sketch_image, marks=None):
    """Background worker. Stage 1 AI, Stage 2 marks, Stage 3 sketch."""
    global _gen_status
    try:
        generate(positive, negative, num_images=num,
                 sketch_image=sketch_image, marks=marks)
        with _gen_lock:
            _gen_status["done"]    = True
            _gen_status["running"] = False
    except Exception as e:
        with _gen_lock:
            _gen_status["error"]   = str(e)
            _gen_status["done"]    = True
            _gen_status["running"] = False


@app.route("/api/generate", methods=["POST"])
@login_required
@role_required("police")
def api_generate():
    """Start generation in a background thread. Returns immediately."""
    global _gen_status
    with _gen_lock:
        if _gen_status["running"]:
```

```
            return jsonify({"error": "A generation is already in progress."}), 409

    positive = request.form.get("positive_prompt", "")
    negative = request.form.get("negative_prompt", "")
    num      = min(int(request.form.get("num_images", 4)), config.MAX_NUM_IMAGES)

    marks_json = request.form.get("marks_json", "[]")
    try:
        marks = json.loads(marks_json) if marks_json else []
    except (json.JSONDecodeError, TypeError):
        marks = []

    if not positive:
        return jsonify({"error": "No prompt provided"}), 400

    sketch_image = None
    if "sketch" in request.files and request.files["sketch"].filename:
        from PIL import Image as PILImage
        sketch_file = request.files["sketch"]
        if allowed_image(sketch_file.filename):
            sketch_image = PILImage.open(sketch_file).convert("RGB")

    # Clear previous run's images so polling starts fresh.
    for f in config.GENERATED_DIR.glob("generated_*.png"):
        try: f.unlink()
        except Exception: pass

    with _gen_lock:
        _gen_status = {"running": True, "done": False, "error": None,
                       "num": num,  "mode": "text"}

    threading.Thread(target=_run_generation_job,
                     args=(positive, negative, num, sketch_image, marks),
                     daemon=True).start()

    return jsonify({"started": True, "num_images": num,
                    "marks_count": len(marks)})


@app.route("/api/generate-status", methods=["GET"])
@login_required
@role_required("police")
def api_generate_status():
    """Poll generation progress. Returns URLs of images saved so far."""
    with _gen_lock:
        status = dict(_gen_status)

    # Parallel API mode finishes images out of order — collect every
    # slot that exists on disk. Append mtime to bust the browser cache
    # when Stage 3 rewrites a slot (photo → pencil-sketch variant).
    images = []
    for i in range(1, status.get("num", 4) + 1):
        fname = f"generated_{i}.png"
        fpath = config.GENERATED_DIR / fname
        if fpath.exists():
            mtime = int(fpath.stat().st_mtime)
            images.append(url_for("serve_generated",
                                  filename=fname) + f"?t={mtime}")
```

```
    return jsonify({
        "running": status["running"],
        "done":    status["done"],
        "error":   status["error"],
        "count":   len(images),
        "total":   status["num"],
        "images":  images,
    })


# ═══════════════════════════════════════════════════════════
# Step 4 — match selected face against the criminal DB
# ═══════════════════════════════════════════════════════════

@app.route("/api/match", methods=["POST"])
@login_required
@role_required("police")
def api_match():
    data           = request.get_json()
    image_filename = data.get("image_filename", "")
    attributes     = data.get("attributes")

    if not image_filename:
        return jsonify({"error": "No image filename"}), 400

    image_path = config.GENERATED_DIR / secure_filename(image_filename)
    if not image_path.exists():
        return jsonify({"error": "Image not found"}), 404

    try:
        matcher = get_matcher()
        results = matcher.match(str(image_path),
                                query_attributes=attributes,
                                top_k=config.TOP_K_MATCHES)

        # Enrich with full DB record for the detail panel.
        enriched = []
        conn = get_db()
        for r in results:
            criminal_name = r.get("name", "")
            row = conn.execute(
                "SELECT * FROM criminals WHERE criminal_code=? OR name=?",
                (criminal_name, criminal_name)).fetchone()
            entry = dict(r)
            if row:
                entry["db_record"] = {
                    "id":                 row["id"],
                    "criminal_code":      row["criminal_code"],
                    "name":               row["name"],
                    "age":                row["age"],
                    "crime_type":         row["crime_type"],
                    "crime_year":         row["crime_year"],
                    "no_of_crimes":       row["no_of_crimes"],
                    "last_known_location": row["last_known_location"],
                    "status":             row["status"],
                    "description":        row["description"],
                    "face_url":           (url_for("serve_criminal",
                                                   filename=row["face_image_path"])
                                           if row["face_image_path"] else None),
```

```
                }
            enriched.append(entry)
        conn.close()
        return jsonify({"matches": enriched})
    except Exception as e:
        traceback.print_exc()
        return jsonify({"error": str(e)}), 500


# ═══════════════════════════════════════════════════════════
# Save pipeline run to a case for traceability + audit trail
# ═══════════════════════════════════════════════════════════

@app.route("/api/save-run", methods=["POST"])
@login_required
@role_required("police")
def api_save_run():
    data = request.get_json()

                        case_number      =      f"CASE-{datetime.now().strftime('%Y%m%d')}-
{uuid.uuid4().hex[:4].upper()}"
    conn = get_db()
    cur = conn.execute(
        "INSERT INTO cases (case_number, officer_id, incident_location, status) "
        "VALUES (?,?,?,?)",
        (case_number, current_user.id, data.get("incident_location", ""), "open"))
    case_id = cur.lastrowid

    conn.execute(
        "INSERT INTO investigation_runs "
        "(case_id, officer_id, audio_file_path, transcript, stt_language, "
        "stt_confidence, description_text, parsed_attributes, positive_prompt, "
        "negative_prompt, generated_image_paths, generation_mode, "
        "had_sketch_input, selected_image_path, match_results, "
        "top_match_name, top_match_score) "
        "VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)",
        (case_id, current_user.id,
         data.get("audio_path"), data.get("transcript"),
         data.get("stt_language"), data.get("stt_confidence"),
         data.get("description_text"),
         json.dumps(data.get("attributes", {})),
         data.get("positive_prompt"), data.get("negative_prompt"),
         json.dumps(data.get("generated_images", [])),
         data.get("generation_mode", "text"),
         1 if data.get("had_sketch") else 0,
         data.get("selected_image"),
         json.dumps(data.get("match_results", [])),
         data.get("top_match_name"), data.get("top_match_score")))
    conn.commit()
    conn.close()

    audit("run_pipeline", "case", case_id, {
        "case_number": case_number,
        "top_match":   data.get("top_match_name"),
        "score":       data.get("top_match_score"),
    })
    return jsonify({"status": "ok", "case_id": case_id,
                    "case_number": case_number})
```

```
if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000, debug=False)
```